프로젝트 전체 기술 감사 보고서

> **상태: 부분 감사 (백엔드 전면 + 프론트엔드·인프라 일부).** 아래 §0의 범위 표를 먼저 확인할 것. 이 문서는 **어떤 코드도 수정하지 않았고**, 테스트를 **실행하지 않았다.** 모든 항목은 소스를 직접 읽고 확인한 사실과, 확인하지 못해 "후보/가설"로 표시한 항목으로 구분했다.

항목	내용
대상	`github.com/mingming-01/maple-growth-tracker`
기준 ref	기본 브랜치 HEAD, 커밋 `259acf6` (다른 작업 브랜치의 변경은 반영되지 않음)
감사일	2026-11-10
제출 위치	이 파일을 `docs/PROJECT_FULL_AUDIT_REPORT.md`로 배치 (저장소에는 아직 커밋하지 않음)
라인 번호	조회 도구가 줄 번호를 제공하지 않아 **함수·클래스명**으로 위치를 표기

---

0. 감사 범위와 방법

구분	내용
디렉터리·파일 목록	전 디렉터리 열거 완료 (열거된 파일 약 120개, `static/images`·`tests/fixtures` 내부는 미열거)
본문을 끝까지 읽은 파일 (36개)	`app.py`, `tracker/{application,config,runtime}.py`, 컨트롤러 6개 전부, 서비스 7개 전부(`__init__` 제외: `admin_log_service`, `ai_analysis_service`, `ai_text_service`, `character_detail_service`, `growth_history_service`, `nexon_service`, `request_limits`), `domain/` 2개, `integrations/` 3개, `repositories/` 2개, `frontend/src/{App.jsx, services/api.js, lib/storage.js}`, `vite.config.js`, 루트·프론트 `package.json`, `requirements.txt`, `.gitignore`, `.github/workflows/*.yml` 2개, `migrations/20261027_growth_daily_history.sql`, `docs/P5_LEGACY_CLEANUP_AUDIT.md`
해시 비교만 수행	`static/js/growth.js` ↔ `frontend/src/lib/growth.js` (Git blob SHA 비교)
읽지 못한 영역	`static/app.js` (117KB), `static/js/character-details.js` (62KB), `static/style.css`, `templates/*`, 프론트 컴포넌트 전부(`GrowthDashboard.jsx` 39KB, `AnalyticsAndComparison.jsx` 25KB, `CharacterDetails*.jsx` 약 70KB, `GrowthHome` 등), `frontend/src/styles.css`·`react-compat.css`, 테스트 본문(Python 23개·JS 4개·E2E 4개), `README.md` (29KB), 나머지 `docs/` 8개, `migrations/20261106_api_snapshots.sql`, `scripts/run_e2e_server.py`, `playwright.config.cjs`
실행	**테스트·빌드·린트 미실행.** 분석 환경의 bash에 네트워크가 없어 클론과 실행이 불가능했다.
정적 참조 검증	저장소 전체 grep을 하지 못했다. 따라서 **죽은 코드는 모두 "후보"**이며 삭제 가능으로 판정하지 않는다.

---

1. Executive Summary

전체 상태. 백엔드는 계층 분리(controller / services / domain / integrations / repositories)와 외부 API 장애 대응(429 즉시 반환, 제한적 재시도, 스냅샷 폴백, 스레드 풀 상한)이 의도적으로 설계되어 있고, 보안 기본기(CSRF, 해시 비밀번호, 로그인 잠금, 세션 쿠키 옵션, 64KB 본문 제한, 로그에서 요청 본문 배제)도 갖췄다. 반면 React 전환 이후의 **성능 구조, 중복 구현, 레거시 공존, 설정·배포 재현성**에 기술 부채가 집중되어 있다.

가장 심각한 문제 (확인된 것)

1. 검색 경로가 직렬 호출과 요청마다 새 DB 연결(비풀링)로 구성되어 있어 느릴 수밖에 없는 구조다 (PERF-01).
2. `GET /api/ai-analysis`의 레거시 분기가 프롬프트의 섹션 이름과 검증기의 섹션 이름이 달라 **항상 검증 실패**한다 (BUG-01).

가장 심각한 문제 (검증이 필요한 후보)

- Render 프록시 뒤에서 IP 기반 제한(AI 한도·관리자 잠금)이 사실상 전체 공용 키로 동작할 가능성 (SEC-01).
- 배포 빌드 절차가 저장소에 없고, React 빌드가 없으면 `/`가 조용히 레거시 화면으로 폴백 (OPS-01).
- 사용자가 보고한 랭킹 증상(직업·월드 순위가 월드 순위와, 직업 순위가 종합 순위와 같게 표시)의 원인 후보 (H-01).

가장 큰 기술 부채. (a) `runtime.py` 만능 모듈과 하위 호환 코드, (b) 동일 로직의 반복(관리자 로그 2벌, 히스토리 컨트롤러 내부 반복, AI 폴백 2벌, DB 부트스트랩 3벌, `growth.js` 2벌), (c) 레거시 화면이 React 화면의 CSS 의존성까지 물고 있는 공존 구조.

개선 방향. 안정성(Phase 1) → 중복·후보 정리(Phase 2) → 책임 분리(Phase 3) → 성능(Phase 4) 순. 랭킹 정확성은 속도보다 우선한다.

집계 (확인 범위 한정) — P0: 0 / P1: 4 / P2: 24 / P3: 24 / 합계 52, 별도 가설 1건. 중복 10건, 죽은 코드 후보 8건. 상세는 §11.

---

2. 프로젝트 구조 및 아키텍처 분석

2.1 전체 구조

```
app.py                    WSGI 진입점 (+ sys.modules 교체 호환 코드)
tracker/
  application.py          앱 팩토리, Blueprint 6개 등록
  config.py               환경변수 → Config
  runtime.py              ★ 상태/캐시/로그DB/스냅샷/에러핸들러 혼재 (19KB)
  controller/             page, admin, character, character_detail, history, ai
  services/               nexon, character_detail, ai_analysis, ai_text, request_limits,
admin_log, growth_history
  domain/                 growth_calculations, ai_growth_insights
  integrations/           nexon_api, http_client, ai_provider
  repositories/           growth_history_store, api_snapshot_repository
frontend/                 React 19 + Vite 6 (빌드 → static/react-preview/, gitignore)
static/, templates/       레거시 화면 (app.js 117KB, style.css 97KB, index.html 36KB)
tests/ tests-js/ e2e/     Python 23, JS 4, Playwright 4
```

2.2 핵심 실행 흐름 (캐시 미적중, 검색 1회)

1. 브라우저 `GET /api/character` → `resolve_character` (Nexon `id`, `character/basic` 2회)
2. `get_character_rankings`: 종합 순위 1회 → 월드/직업월드/직업 3회 병렬(스레드풀) → 날짜 폴백 시 최대 3일 반복
3. `get_current_combat_power`: `character/stat` 1회 (프로필에서도 스탯 API를 호출)
4. `record_best_combat_power`: 스냅샷 DB 읽기 + 조건부 쓰기
5. `load_api_snapshot` + `save_api_snapshot`
6. `after_request`의 `log_action`: 로그 DB 새 연결 1회
7. 브라우저는 응답 후에야 `GET /api/history`를 호출 (`App.jsx::searchName`이 `await`로 직렬화)

2.3 구조적 문제 (요약)

- `runtime.py`가 DB 로그, 스냅샷 위임, 캐시, AI 응답 캐시, 에러 핸들러, 상태 컨테이너를 모두 담는다 (STR-01).
- 컨트롤러 간 직접 호출 (`ai_controller` → `history_controller.get_history()`) (STR-02).
- `history_controller.py`(27KB)에 비즈니스 로직이 있다 (STR-03).
- 레거시 화면과 React 화면 공존, React가 `static/style.css`에 의존 (STR-04, P5 문서로 확인).
- 설정 접근 방식 3종 혼재: `RuntimeState`(app.config 스냅샷), `_setting()`(app 컨텍스트 또는 `Config` 클래스), `current_app.config` 직접 접근 (DUP-06).

---

3. 반복 코드 및 중복 구현 분석

ID	중복 위치	역할	통합 가능성	통합 시 이점	위험	우선순위
DUP-01	`static/js/growth.js` = `frontend/src/lib/growth.js` (blob SHA 동일, 14,056B)	성장 지표 계산	가능하나 **레거시가 `static/app.js` 에서 import 중** (P5 문서)	계산 로직 1벌	레거시 경로 회귀. 레거시 폐기 승인 전에는 삭제 불가, 단일 소스화(빌드 복사 등)만 가능	P2
DUP-02	`admin_controller.admin_logs` / `admin_logs_api`	로그 필터·조회	가능 (쿼리 빌더 함수 추출)	약 100줄 감소, 화면·API 불일치 방지	HTML/JSON 응답 경고 문구가 약간 다름(통합 시 보존 필요)	P2
DUP-03	`history_controller.get_history` : "재확인 필요" 판정식 3회(`fetch_history` , `missing_dates` , `remainingDays`), 저장 기록→응답 dict 변환 4회	기록 상태 판정·변환	가능 (헬퍼 2개)	판정 규칙이 갈라질 위험 제거	판정 조건 변경 시 `not_ready` 재시도 정책에 영향	P2
DUP-04	`fetch_history` (7일) ↔ `fetch_historical_record` (30/90일)	랭킹→기본정보 폴백으로 일일 스냅샷 생성	부분 가능	폴백 규칙 일원화	두 경로의 정책 차이(7일: 429/5xx에서 기본정보 API 연쇄 호출 금지, 아카이브: 허용)는 **의도된 차이일 수 있어 그대로 유지 필요**	P2
DUP-05	`ai_analysis_service` : Gemini→OpenRouter 폴백 블록이 분석·비교에 2벌, 히스토리 필드 화이트리스트가 서비스 2곳 + `ai_controller` 에 3벌	AI 호출 오케스트레이션	가능	폴백 정책 단일화	분석은 품질 검증(`validate_growth_report`), 비교는 검증 없음 — 통합 시 이 차이를 파라미터로 보존	P2
DUP-06	`_setting()` 이 `nexon_api.py` , `ai_provider.py` , `request_limits.py` 에 3벌	설정 접근	가능	설정 경로 1개	스레드 풀 작업자는 앱 컨텍스트가 없어 `Config` 폴백을 쓰는 동작이 현재 암묵적으로 의존됨	P2

ID	중복 위치	역할	통합 가능성	통합 시 이점	위험	우선순위
DUP-07	Postgres 연결·초기화·RLS/REVOKE 블록 3곳(`runtime` , `growth_history_store` , `api_snapshot_repository`), `KOREA_TIMEZONE` 3곳(`runtime` , `admin_log_service` , `nexon_service` 인라인), 백엔드 판별식 3곳	DB 부트스트랩	가능	풀링·SSL 옵션·역할 가정을 한곳에서 관리	로그/스냅샷은 폴백(SQLite) 정책, 히스토리는 예외 전파 정책으로 **현재 동작이 서로 다름**	P2
DUP-08	스키마 정의가 `growth_history_store.initialize()` 와 `migrations/*.sql` 에 이중 존재	테이블 정의	가능	단일 소스	두 정의가 현재 동일해 보이나 (읽은 마이그레이션 기준) 두 번째 마이그레이션 (`api_snapshots`)은 대조 못 함	P2
DUP-09	기간별 AI 지표 선별: `api.js::selectAiMetricsForPeriod` ↔ `ai_analysis_service._build_growth_report_prompt`	30/90일에 7일 지표 제외	서버 쪽 한 곳으로	규칙 일원화	클라이언트가 서버 캐시 키(프롬프트)에 영향	P3
DUP-10	일별 변화 계산 3벌: `calculate_recent_history_changes` , `calculate_archive_history_changes` , `_period_summary` (+ JS `growth.js`)	경험치/랭킹 변화	**규칙이 서로 다름** (최근 7일은 `recordStatus` 무시·레벨업 시 변화 None, 아카이브는 둘 다 available일 때만, 인사이트는 레벨 불변 구간만)	기본 연산만 공통화 가능	**의도된 차이를 지우면 지표가 바뀜 → 통합보다 유지 권장**	P3

4. 죽은 코드 및 불필요한 리소스 분석

> 모두 **후보.** 저장소 전체 grep, 테스트 참조, 동적 접근을 확인하지 못했다. 삭제 근거로 쓰지 말 것.

ID	후보	확인된 사실	판정 보류 사유
DEAD-01	`runtime.py` 미사용 import: `redirect`, `url_for`, `session`, `render_template`, `wraps`, `re`, `date`, `timedelta`, `admin_required`, `format_admin_log_time`	파일 본문에서 사용 흔적 없음	`app.py`가 `sys.modules`를 `runtime`으로 교체해 **옛 테스트·스크립트가 `app.<이름>`으로 접근**하므로 재노출 목적일 수 있음
DEAD-02	`runtime.py` 하단의 고아 주석("# 메인 화면", "# 관리자 로그인 및 로그 조회", "# 캐릭터 정보와 현재 랭킹 조회")	해당 코드 없음	없음(주석, 안전)
DEAD-03	`ai_controller.ai_analysis`의 `if ocid:`	바로 위에서 `ocid` 존재가 이미 보장됨 → 항상 참	없음
DEAD-04	`request_nexon_api(..., historical=False)`의 `historical` 인자	함수 본문에서 미사용, 호출부는 계속 전달	인터페이스 호환
DEAD-05	`RuntimeState.nexon_request_lock`, `last_nexon_request_at`, `nexon_request_count`	읽은 파일에서 쓰기만 있고(`_throttle_nexon_request`) 읽기 없음, 락은 미사용(모듈 전역 락 사용)	테스트·관리자 화면이 읽을 가능성
DEAD-06	`character_detail_service._fetch_champion_image` ("기존 호출부 호환성")	읽은 코드 내 호출 없음	테스트 참조 가능
DEAD-07	`nexon_api.get_ranking_page`	`nexon_service`가 import하지 않음	테스트(`test_nexon_api_ranking.py`) 참조 가능
DEAD-08	인덱스 `idx_growth_daily_name_date (character_name, snapshot_date)`	읽은 쿼리는 모두 `ocid` 기준(PK가 커버)	읽지 못한 코드나 운영 수동 쿼리가 이름으로 조회할 가능성. 운영 DB에서 사용 통계로 확인 필요

삭제하면 안 되는 것으로 확인된 것: `static/jobs.json` — 프론트 코드에서 참조되지 않지만 `nexon_service._jobs_path()`가 사용하며, 삭제 시 테스트가 실패함이 `docs/P5_LEGACY_CLEANUP_AUDIT.md`에 기록되어 있다. 레거시 템플릿·JS·CSS·이미지도 같은 문서의 근거(현재 실행 경로 존재, README의 운영 검증 선행 조건)에 따라 유지.

---

5. 백엔드 분석 결과

5.1 상세 기록 (P1 및 핵심 P2)

[BUG-01] 레거시 `GET /api/ai-analysis` 분기가 항상 검증 실패

- **분류:** 버그 / **우선순위:** P1 (코드상 확정, 호출 경로 미확인)
- **위치:** `ai_controller.ai_analysis` (라우트 `methods=["GET","POST"]`), `ai_analysis_service.generate_ai_analysis`의 `else:` 프롬프트, `ai_text_service.validate_growth_report`
- **현황:** GET이면 `metrics=None`이라 `computed_metrics is None` 분기의 프롬프트가 쓰인다. 이 프롬프트는 `[성장 패턴] [성장 분석] [레벨업] [AI 판단]` 4개 섹션을 요구한다.
- **문제점:** `validate_growth_report`는 `[성장 패턴] [성장 변화] [레벨업 분석] [성장 전망] [AI 코멘트]` 5개를 모두 요구한다. 모델이 프롬프트를 따르면 항상 검증 실패 → 502(`invalid_model_output`). AI 호출과 일일 한도만 소모한다.
- **근거:** 두 파일의 상수·프롬프트 문자열 대조.
- **영향 범위:** GET으로 호출하는 클라이언트(읽지 못한 `static/app.js`가 사용할 가능성).
- **개선 방향:** GET 분기를 제거하거나(POST 전용) 프롬프트·검증기를 같은 섹션 정의에서 생성. 레거시 화면 사용 여부 확인 후 결정.
- **예상 효과:** 불필요한 AI 호출·한도 소모 제거.

- **위험 요소:** 레거시 화면이 GET에 의존하면 회귀.

[PERF-01] 검색 경로의 직렬 호출과 요청당 새 DB 연결

- **분류:** 성능 / **우선순위:** P1
- **위치:** `character_controller.get_character`, `runtime.log_action`, `api_snapshot_repository.*`, `App.jsx::searchName`
- **현황/근거:** §2.2의 흐름. 캐시 미적중 시 Nexon 호출이 최소 7회(`id`, `basic`, 종합 순위, 순위 3, `stat`; 날짜 폴백 시 날짜당 4회 추가), 이어서 `/api/history`가 최대 7일 × 날짜당 1~3회. 모든 Nexon 호출은 전역 락 안에서 시작 간격을 0.22초로 제한한다(PERF-02). Postgres 사용 시 `get_character` 1회에 새 SSL 연결이 약 3~5회(스냅샷 읽기/쓰기, 최고 전투력 읽기/쓰기, 로그), `get_history`에도 약 4~5회.
- **문제점:** 사용자가 보고한 "스펙 상세 기능 추가 후 검색이 느려짐"과 구조적으로 부합한다(프로필 응답이 스탯 API와 최고 전투력 스냅샷 쓰기까지 기다림). **실제 소요 시간은 측정하지 못했다.**
- **영향 범위:** 모든 신규 검색.
- **개선 방향:** (1) 프로필에서 스탯·스냅샷 쓰기를 분리(응답 후/지연 로드), (2) 검색과 히스토리를 병렬 시작 또는 프로필 선응답, (3) DB 연결 풀, (4) `requests.Session`. 이미 세운 6단계 성능 계획과 방향이 일치한다.
- **예상 효과:** 체감 응답 단축.
- **위험 요소:** 랭킹 정확성이 우선이므로 랭킹 호출 구조는 건드리지 말고 **스탯·DB 쓰기 쪽부터** 분리할 것.

[SEC-01] 프록시 뒤 `request.remote_addr` 기반 제한 (후보, 검증 필요)

- **분류:** 보안/안정성 / **우선순위:** P1 후보
- **위치:** `request_limits._client_key`, `ai_controller._ai_rate_limit_response`, `admin_controller.admin_login`; `application.create_app`에 `ProxyFix` 없음(확인)
- **현황:** AI 분당·IP별 일일 한도와 관리자 로그인 잠금이 `request.remote_addr`로 키를 만든다.
- **문제점:** Render 같은 리버스 프록시 뒤에서 별도 설정이 없으면 모든 방문자의 `remote_addr`가 프록시 주소로 보일 수 있다. 그러면 IP별 한도가 전체 공용이 되고(전체 일일 한도 100과 별개로 IP 20이 전체 합산), 관리자 로그인 5회 실패가 모든 사람을 잠근다.
- **근거:** 코드상 `ProxyFix` / `X-Forwarded-For` 처리 없음. gunicorn 시작 옵션과 Render 설정은 저장소에 없어 **런타임 확인 불가**.
- **개선 방향:** 배포 환경에서 `remote_addr`를 실제로 로그로 확인한 뒤, 신뢰할 프록시 홉 수를 지정한 `ProxyFix` 적용.
- **위험 요소:** 신뢰 홉 수를 잘못 지정하면 클라이언트가 헤더로 IP를 위조해 한도를 우회.

[OPS-01] 빌드·배포 절차가 저장소에 없음 (후보, 검증 필요)

- **분류:** 설정/배포 / **우선순위:** P1 후보
- **근거:** `static/react-preview/`는 `.gitignore` 대상이며 `render.yaml`, `Procfile`, `Dockerfile`이 루트에 없다. `page_controller.home`은 빌드 산출물이 없으면 **오류 없이** 레거시 `index.html`로 폴백한다. 배포 URL의 응답은 메타 정보만 확인되어 어느 화면이 서빙되는지 판별하지 못했다.
- **문제점:** Render 대시보드의 빌드 명령에 `npm --prefix frontend run build`가 없으면 React 대신 레거시가 조용히 서빙된다. 시작 명령(워커 수 등)도 문서화 여부를 확인하지 못했다.
- **개선 방향:** 빌드·시작 명령을 `render.yaml`로 코드화하고, 빌드 부재 시 운영에서는 폴백 대신 경고/실패 처리.

[H-01] 랭킹 증상의 원인 후보 (가설, 미확정)

- 사용자 보고: 직업·월드(374위)와 직업 전체(2,278위)가 각각 월드·종합 순위와 같은 값으로 표시.
- **읽은 코드에서 관찰한 것:** `get_character_rankings`는 4개 순위를 각각의 응답에서 뽑고, `_validate_filtered_entry`는 응답 항목의 `class_name` / `sub_class_name` / `world_name`만 검증한다. `_rank_is_consistent`는 "하위 순위 ⩽ 상위 순위"만 검사하므로 **동일 값은 통과**한다. 필터 파라미터가 무시되어 비필터 항목이 돌아오면(직업 메타데이터가 일치하는 한) 이 검증은 감지하지 못한다.
- **미확인:** Nexon API가 `ocid` + `class` 조합에서 실제로 무엇을 반환하는지, 프론트가 값을 어떻게 매핑하는지(읽지 못함).
- **검증 방법:** 문제 캐릭터로 응답의 `rankingApiDiagnostics`(상태코드·`returnedEntries`·`filterUsed`)와 4개 호출의 원본 항목을 비교. `ranking.get_ranking`의 `params`에 `class`가 실제로 전달되는지 확인.

기타 P2 상세 (요약 형식)

- **BUG-02** `_fetch_champion_profile` : 실패 시 빈 `{}` 를 12시간 캐시한다 → 일시적 오류가 반나절 고정.
- **BUG-03** `http_client.request_with_retries` : 408/425/5xx의 `Retry-After` 를 상한 없이 `sleep` 한다("do not shorten" 주석). 큰 값이 오면 동기 워커가 그만큼 점유됨. (429는 즉시 반환하도록 되어 있어 해당 없음)
- **SEC-02** `/api/character` , `/api/history` , `/api/character-details` 에 요청 제한이 없고 `refresh=1` 로 서버 캐시를 우회할 수 있다. `GET /api/history/archive?collect=1` 은 **인증 없는 GET이 업스트림 호출과 DB 쓰기를 유발**한다 (이름-OCID 일치 검증과 배치 10일 제한으로 완화됨).
- **SEC-03** `REVOKE ... FROM PUBLIC, anon, authenticated` 가 3곳에 있어 Supabase 역할 존재를 가정한다. 다른 Postgres에서는 실패하며 **로그는 조용히 SQLite로 전환, 스냅샷도 SQLite 전환, 히스토리 저장소는 호출마다 예외**로 동작이 제각각이다.
- **SEC-04** 한도 상태가 프로세스 메모리이므로 gunicorn 워커 수만큼 한도가 배가되고, `ai_request_timestamps` · `admin_login_failures` 는 키를 정리하지 않아 계속 쌓인다.
- **PERF-02** `_throttle_nexon_request` 가 **락을 쥔 채** `sleep` → 모든 사용자의 모든 Nexon 호출이 시작 시점에서 직렬화 (0.22초). 스레드풀 병렬화의 이득이 제한된다.
- **PERF-03** `requests.get` 직접 호출로 Session/연결 재사용이 없다(매 호출 TCP+TLS).
- **PERF-04** 모든 DB 작업이 새 연결을 연다(`sslmode=require` , 풀 없음). 3개 모듈에 동일 패턴.
- **PERF-05** `_SOURCE_CACHE` · `_CHAMPION_IMAGE_CACHE` 는 크기 상한이 없고(런타임 캐시는 1000 상한), `_fetch_source` 캐시 키에 `json.dumps(spec)` 전체가 들어간다.
- **PERF-06** `api_snapshots` 테이블에 보존 기간·정리 로직이 없다(캐릭터·히스토리 JSON, `combat-power-best:*` 키가 계속 누적).

5.2 P3 요약

- BUG-04 AI 한도가 캐릭터 유효성 확인 **이전**에 차감된다(오타 입력도 한도 소모).
- BUG-05 `admin_required` 가 JSON API(`/admin/api/logs`)에도 302 리다이렉트를 반환(프론트가 `redirected` 검사로 우회).
- BUG-06 `should_use_openrouter` 가 오류 문자열에서 "500", "503" 등 부분 문자열을 찾는다(관련 없는 숫자에서 오탐 가능).
- SEC-05 `generate_ai_*` · `ai_analysis` 가 예외 원문을 `print` (다른 곳의 "민감정보 미출력" 정책과 불일치).
- SEC-06 `app.py` 의 `app.run(debug=True, host="0.0.0.0")` (개발용 `npm run dev:backend` 가 이 경로) — 개발 PC 네트워크에 Werkzeug 디버거 노출. 비운영 `ADMIN_PASSWORD` 평문 폴백(운영에서는 차단됨, 확인).
- SEC-07 React 관리자 로그인이 로그인 HTML을 파싱해 CSRF를 추출한다(JSON 로그인 엔드포인트 부재, 마크업 변경에 취약).
- SEC-08 클라이언트가 보낸 `metrics` / `periodSummary` 가 검증 없이 프롬프트에 들어간다(64KB 제한·한도로 완화, 영향은 본인 응답에 한정).
- CFG-01 Gemini 모델명 `gemini-3.6-flash` 가 코드에 하드코딩(설정 불가). OpenRouter 폴백이 있어 **모델명이 잘못돼도 겉으로는 드러나지 않을 수 있음**. 모델명의 유효성은 확인하지 못했다.

---

6. 프론트엔드 분석 결과 (제한적)

읽은 것: `App.jsx` , `services/api.js` , `lib/storage.js` , `vite.config.js` , `package.json` . 나머지 컴포넌트는 **미검토**.

- STR-07 `api.js` 의 `getCharacter` / `getHistory` 는 `refresh = true` 가 **기본값**이다. 현재 `App.jsx` 는 항상 `{refresh}` (기본 false)를 넘겨 문제가 없지만, 다른 호출부가 옵션을 생략하면 서버 캐시가 전부 우회된다(호출부 미확인).
- STR-08 `App.jsx` : `onStatsLoaded` 가 `setCharacter` 업데이터 함수 안에서 `saveTrackerState` (부수효과)를 호출, `useEffect` 와 수동 저장이 중복, `saveTrackerState` 의 실패 반환값 무시, 상태 전체를 매번 읽고-병합-쓰기(90일 기록 포함 가능).
- `searchName` 은 `getCharacter` 완료 후 `getHistory` 를 호출한다(OCID 의존). 병렬화하려면 이름 기반 경로가 필요하다 (§5 PERF-01).

- 번들: `App.jsx`가 `AdminDashboard`, `GrowthDashboard`(`loadGrowthPeriod`)를 정적 import → 관리자·성장 화면 코드가 초기 번들에 포함될 구조. (`html2canvas`·`jspdf`는 동적 import로 이미 분리됨.) 실제 번들 크기는 측정하지 못했다.
- STR-06 운영 화면의 경로·산출물 이름이 `react-preview`(`/preview/`, `static/react-preview/`)로 남아 있다.
- 푸터에 "레거시 관리자 로그인" 링크가 노출된다(의도 여부 확인 필요).
- 접근성: 헤더 드롭다운에 `aria-current`, 검색 입력 `sr-only` 라벨 등 기본 배려가 있다. 그 외 전수 검토는 미실시.

---

7. 데이터베이스 및 데이터 흐름 분석

- 테이블: `growth_daily_snapshots`(PK `ocid, snapshot_date`, `record_status` 체크 제약), `api_snapshots`(`cache_key` PK), `action_logs`.
- 좋은 점: `ON CONFLICT ... WHERE existing <> 'available' OR EXCLUDED = 'available'`로 **정상 스냅샷을 빈 응답이 덮어쓰지 못하게** 한 조건, 파라미터 바인딩 일관 사용(SQL 인젝션 징후 없음, 로그 검색도 바인딩), RLS 활성화.
- 문제: DUP-07/08, SEC-03, PERF-04, PERF-06(스냅샷 무한 증가), DEAD-08(인덱스), 이중 캐시 계층(서버 메모리 5분~6시간 + 스냅샷 DB + 클라이언트 `localStorage` 6시간~30일)의 TTL 정책이 문서화되어 있지 않음.
- 캐시 키가 `character:{입력값}` / `history:{입력값}` 형태라 **대소문자·공백이 다르면 별도 캐시**다(OCID 기준이 아님). (P3, 영향은 캐시 효율)
- 읽지 못한 것: 두 번째 마이그레이션, 실제 운영 DB 스키마/행 수.

---

8. 테스트 및 CI/CD 분석

- 구성: Python `unittest` 23개 파일, Node `--test` 4개, Playwright E2E 4개(최대 36KB 파일), 워크플로 2개. **본문·실행 결과 미확인**이므로 테스트 품질(Mock 적절성, 형식적 테스트 여부)은 평가하지 못했다.
- CI-01 `e2e.yml`: `pull_request`(main)와 수동 실행만, 경로 필터에 `tracker/**`가 없고(`page_controller.py`만 포함) 백엔드 변경이 E2E에 걸리지 않는다. 프론트 의존성을 `npm install`(비결정)로 설치(단위 테스트 잡은 `npm ci`).
- CI-02 `requirements.txt`가 **버전 미고정, 락 파일 없음** → 빌드 재현성 부재(Flask·google-genai 등 메이저 변경에 취약).
- CI-03 린트·포맷·타입 검사·커버리지·의존성 취약점 점검 없음.
- CI-04 CI가 레거시 JS 4개의 구문 검사(`node --check`)를 유지 → 레거시 폐기 시 함께 정리 필요.
- 두 워크플로 모두 `main` 대상이며 다른 브랜치의 푸시에는 단위 테스트가 돌지 않는다(PR 필요).

---

9. 보안 및 안정성 분석

확인된 양호 사항: CSRF 토큰(로그인·로그아웃), `hmac.compare_digest`, 비밀번호 해시 검증과 운영에서 평문 비밀번호 비활성, 로그인 실패 잠금, 세션 쿠키 `HttpOnly/SameSite=Lax/Secure(운영)`, `MAX_CONTENT_LENGTH=64KB`, 전역 예외 핸들러가 내부 정보를 반환하지 않음, 요청 본문을 로그에 저장하지 않음, Jinja 자동 이스케이프 사용 가정(템플릿 미검토).

실제 확인된 문제: SEC-02, SEC-03, SEC-04, SEC-05, SEC-06, SEC-07, SEC-08(§5). **잠재/검증 필요:** SEC-01, OPS-01. **미실시:** 비밀 스캔, 템플릿·레거시 JS의 XSS 검토, 의존성 취약점 점검, 실제 Render 설정 확인.

---

10. 코드 품질 및 유지보수성 분석

- 큰 단위: `history_controller.get_history`(약 330줄, 중첩 클로저 `fetch_history` 약 170줄), `runtime.py` 19KB, `ai_analysis_service.generate_ai_analysis` 약 150줄(프롬프트 조립·캐시·폴백 혼재), `nexon_service.get_character_rankings`(중첩 함수 4개).
- 하드코딩: 모델명, 월드 목록(`get_world_type`의 리부트 월드 집합), TTL 상수 분산(`CHARACTER_CACHE_TTL`/`HISTORY_CACHE_TTL`은 `runtime`, `DETAIL_CACHE_TTL`은 컨트롤러, `_SOURCE_CACHE_TTL`은 서비스).
- 일관성: `calculate_recent_history_changes`는 첫 항목에 `levelUp` 키를 설정하지 않는다(다른 항목은 설정). 소비 측이 `.get("levelUp", False)`로 방어 중.
- STR-05 `GrowthHistoryService`는 저장소의 단순 위임 래퍼(추상화 이득 없음).
- 명명·언어: 주석·메시지는 한국어, 일부 영어 혼재(스타일 수준).
- `ai_growth_insights`의 `date.today()` 폴백은 서버 로컬 날짜(UTC)이며 KST 기준이 아님(기준일이 항상 전달되어 현재 영향 없음).

- `ai_text_service.sanitize_ai_text`는 단어 치환 사전이 크고("데이터"→"기록" 등) 의미 변조 가능성이 있으나, 품질 게이트로서 **유지할 가치가 있음**.

---

11. 발견된 문제 전체 목록

> 위치는 함수·클래스명. "확정"은 읽은 소스에서 직접 확인, "후보/가설"은 §5·§4 참조.

ID	문제	위치	분류	우선순위	권장 조치
BUG-01	레거시 GET AI 분석이 항상 검증 실패(섹션 불일치)	`ai_controller` / `ai_analysis_service` / `ai_text_service`	버그	P1	GET 분기 제거 또는 섹션 정의 단일화
PERF-01	검색 경로 직렬 호출 + 요청당 새 DB 연결	`character_controller.get_character`, `App.jsx`	성능	P1	스탯·쓰기 분리, 풀, Session
SEC-01	프록시 뒤 `remote_addr` 키(후보)	`request_limits`, 컨트롤러	보안	P1(후보)	운영 `remote_addr` 확인 후 `ProxyFix`
OPS-01	빌드·배포 절차 미코드화, 레거시 무음 폴백(후보)	`.gitignore`, `page_controller.home`	설정	P1(후보)	`render.yaml` 도입, 폴백 경고
BUG-02	챔피언 조회 실패를 12시간 캐시	`_fetch_champion_profile`	버그	P2	실패 결과는 캐시하지 않거나 짧은 TTL
BUG-03	`Retry-After` 무상한 sleep	`http_client.request_with_retries`	버그	P2	상한 적용
SEC-02	비AI 엔드포인트 무제한 + `refresh=1` 우회 + GET 쓰기	컨트롤러	보안	P2	공통 레이트 리밋, 수집을 POST로
SEC-03	Supabase 역할 가정(REVOKE), 실패 시 동작 불일치	3개 DB 모듈	안정성	P2	역할 존재 확인, 정책 통일
SEC-04	프로세스 로컬 한도, 키 무한 증가	`request_limits`	안정성	P2	정리 로직, 워커 수 명시
PERF-02	락을 쥔 채 sleep하는 전역 스로틀	`_throttle_nexon_request`	성능	P2	슬롯 예약 방식
PERF-03	HTTP 세션 미사용	`http_client`	성능	P2	`requests.Session`
PERF-04	요청당 DB 연결, 풀 없음	`runtime`, `growth_history_store`, `api_snapshot_repository`	성능	P2	연결 풀
PERF-05	무상한 모듈 캐시, 큰 캐시 키	`character_detail_service`	성능	P2	상한·LRU, 키 단순화
PERF-06	`api_snapshots` 무한 증가	`api_snapshot_repository`	성능	P2	보존 정책
DUP-01	`growth.js` 바이트 동일 2벌	`static/js`, `frontend/src/lib`	중복	P2	단일 소스화(레거시 폐기 전까지 삭제 금지)
DUP-02	관리자 로그 조회 2벌	`admin_controller`	중복	P2	쿼리 빌더 추출
DUP-03	기록 상태 판정·변환 반복	`history_controller`	중복	P2	헬퍼 추출

ID	문제	위치	분류	우선순위	권장 조치
DUP-04	일일 스냅샷 폴백 로직 2벌	`history_controller`	중복	P2	정책 차이 보존하며 부분 통합
DUP-05	AI 폴백·화이트리스트 반복	`ai_analysis_service`, `ai_controller`	중복	P2	공통 함수
DUP-06	`_setting()` 3벌, 설정 접근 3방식	`nexon_api`, `ai_provider`, `request_limits`	중복	P2	단일 설정 접근
DUP-07	DB 부트스트랩·타임존·판별 3벌	3개 모듈	중복	P2	공통 모듈
DUP-08	스키마 이중 정의	`growth_history_store`, `migrations/`	중복	P2	단일 소스
STR-01	`runtime.py` 만능 모듈, `sys.modules`·monkeypatch 호환	`runtime`, `app`, `application`	구조	P2	책임별 분리, 호환 코드 일몰
STR-02	컨트롤러가 컨트롤러 호출	`ai_controller.ai_analysis`	구조	P2	서비스로 이동
STR-03	컨트롤러에 비즈니스 로직	`history_controller`	구조	P2	서비스 계층 이동
STR-04	레거시 공존, React가 `static/style.css` 의존	`page_controller`, `frontend/index.html`	구조	P2	CSS 이관 후 레거시 단계적 폐기
CI-01	E2E 범위·설치 방식	`e2e.yml`	CI	P2	경로 확장, `npm ci`
CI-02	Python 의존성 미고정	`requirements.txt`	CI	P2	핀/락 파일
BUG-04	한도가 유효성 확인 전에 차감	`ai_controller`	버그	P3	순서 조정
BUG-05	JSON API에 302 리다이렉트	`admin_required`	버그	P3	API는 401 JSON
BUG-06	오류 문자열 부분일치 폴백 판정	`should_use_openrouter`	버그	P3	상태 코드 기반만
SEC-05	예외 원문 print	AI 서비스·컨트롤러	보안	P3	정책 통일
SEC-06	`debug=True, host=0.0.0.0`, 비운영 평문 비밀번호	`app.py`, `admin_controller`	보안	P3	127.0.0.1 기본
SEC-07	관리자 로그인 HTML 파싱	`api.js::loginAdmin`	보안	P3	JSON 엔드포인트
SEC-08	클라이언트 지표를 프롬프트에 무검증 삽입	`ai_controller`	보안	P3	스키마 검증

ID	문제	위치	분류	우선순위	권장 조치
DUP-09	기간별 지표 선별 클라이언트·서버 중복	`api.js`, `ai_analysis_service`	중복	P3	서버로 일원화
DUP-10	일별 변화 계산 3벌(규칙 상이)	`growth_calculations`, `ai_growth_insights`	중복	P3	**유지 권장**, 기본 연산만 공통화
STR-05	단순 위임 서비스	`GrowthHistoryService`	구조	P3	유지/제거 판단
STR-06	운영 화면 명칭 `react-preview`	`page_controller`, `vite.config.js`	구조	P3	명칭 정리
STR-07	`refresh=true` 기본값	`api.js`	구조	P3	기본값 false
STR-08	업데이터 내 부수효과, 저장 중복	`App.jsx`, `storage.js`	구조	P3	정리
DEAD-01	미사용 import 후보	`runtime.py`	죽은 코드	P3	참조 검증 후 정리
DEAD-02	고아 주석	`runtime.py`	죽은 코드	P3	삭제
DEAD-03	항상 참인 조건	`ai_controller`	죽은 코드	P3	삭제
DEAD-04	미사용 `historical` 인자	`request_nexon_api`	죽은 코드	P3	검증 후 제거
DEAD-05	미사용 상태 필드	`RuntimeState`	죽은 코드	P3	참조 검증 후 제거
DEAD-06	호환용 `_fetch_champion_image`	`character_detail_service`	죽은 코드	P3	참조 검증 후 제거
DEAD-07	미사용 `get_ranking_page`	`nexon_api`	죽은 코드	P3	참조 검증 후 제거

ID	문제	위치	분류	우선순위	권장 조치
DEAD-08	이름 기준 인덱스	`growth_history_store`, 마이그레이션	죽은 코드	P3	운영 사용 통계 확인
CI-03	린트·타입·커버리지·취약점 점검 없음	`.github/workflows`	CI	P3	단계적 도입
CI-04	CI의 레거시 JS 구문 검사	`tests.yml`	CI	P3	레거시 폐기 시 정리
CFG-01	모델명 하드코딩	`_generate_gemini_content`	설정	P3	환경변수화
H-01	랭킹 증상 원인 가설(집계 제외)	`nexon_service`	가설	—	§5.1 검증 방법

집계: P0 0 · P1 4 · P2 24 · P3 24 = **52** (가설 1건 별도) / 중복 DUP 10 / 죽은 코드 후보 DEAD 8.

---

12. 개선 권장 로드맵

Phase 1 — 실제 버그·안정성

- 항목: SEC-01 확인 후 `ProxyFix`, OPS-01 빌드/시작 코드화, BUG-01, BUG-02, BUG-03, H-01 검증.
- 선행 조건: 운영 환경에서 `remote_addr`·빌드 로그·랭킹 원본 응답 확인.
- 예상 효과: 제한 정책 정상화, 배포 재현성, AI 한도 낭비 제거.
- 위험: `ProxyFix` 홉 수 오설정, GET 분기 제거 시 레거시 의존.

Phase 2 — 중복·죽은 코드 정리

- 항목: DUP-02, DUP-03, DUP-05, DUP-06, DEAD-02/03(안전한 것부터), DEAD-01/04/05/06/07은 **grep+테스트 검증 후**.
- 선행 조건: 전체 테스트 통과 기준선 확보(미실행 상태).
- 위험: 호환 목적 재노출 import 삭제로 옛 테스트 깨짐.

Phase 3 — 구조·책임 분리

- 항목: STR-01~03, DUP-07/08, STR-04(CSS 이관은 별도 검증).
- 선행 조건: Phase 2 완료, 컨트롤러 단위 회귀 테스트.
- 위험: `app` 모듈 호환 제거로 외부 스크립트 영향.

Phase 4 — 성능·데이터 흐름

- 항목: PERF-01(스탯·쓰기 분리 → 검색/프로필 분리), PERF-03, PERF-04, PERF-02, PERF-05, PERF-06.
- 선행 조건: Phase 3의 DB 접근 일원화(풀 도입 용이), 성능 측정 기준선.
- 위험: **랭킹 정확성 회귀**(속도보다 정확성 우선 원칙 유지), 캐시 불일치.

Phase 5 — 테스트·품질 관리

- 항목: CI-01~04, 의존성 고정, 린트/타입 도입, 읽지 못한 테스트의 품질 감사(Mock 과다·형식적 테스트).
- 위험: 도입 시 기존 코드의 대량 경고.

Phase 6 — 문서화·유지보수 체계

- 항목: README·`docs/` 9개의 현행 코드 일치 점검(이번에 미검토), 배포 가이드, 레거시 폐기 기준(운영 URL 검증 완료 후).
- 위험: 오래된 문서가 현재 구조를 오도.

---

13. 최종 종합 평가 (확인 범위 한정)

항목	평가
기술적 완성도	백엔드는 상당히 높음(장애 대응·스냅샷 정책·검증 설계). 프론트엔드는 평가 불가(대부분 미검토)
유지보수성	중간 이하 — 중복과 `runtime.py`, 큰 컨트롤러
확장성	낮음~중간 — 프로세스 로컬 상태, 연결 비풀링, 전역 스로틀
안정성	중간 — 외부 API 대응은 좋으나 SEC-01·OPS-01 후보와 한도 정책이 미검증
테스트 신뢰도	**평가 불가**(미실행, 본문 미검토). 구성 규모는 양호
기술 부채 수준	중간 — 대부분 구조·중복 부채이며 즉시 장애를 일으키는 P0는 확인되지 않음
당장 수정할 항목	BUG-01, SEC-01 검증, OPS-01 검증, H-01 검증, PERF-01 중 스탯·쓰기 분리
현재는 유지가 나은 항목	`static/jobs.json`, 레거시 화면·자산(운영 검증 전까지), 429 즉시 반환 정책, 스레드풀 상한, "정상 스냅샷을 빈 응답이 덮어쓰지 않는" 저장 조건, `sanitize/validate` 품질 게이트, 관리자 CSRF·잠금, DUP-10(계산 규칙 차이), DUP-04의 정책 차이

---

14. 이번 감사에서 확인하지 못한 사항

- **읽지 못한 파일:** §0의 "읽지 못한 영역" 전부. 특히 프론트 컴포넌트(성능·접근성·상태 관리·재렌더링 평가 불가), 레거시 `static/app.js`·`character-details.js`(BUG-01의 호출 경로, XSS 가능성), 템플릿(자동 이스케이프 여부), README 및 8개 문서(문서-구현 불일치 평가 불가).
- **실행하지 못한 것:** 모든 테스트(Python/JS/E2E), 프론트 빌드, 린트, 비밀 스캔, 의존성 취약점 점검. 사유: 분석 환경에 네트워크가 없어 클론·설치 불가.
- **검증하지 못한 외부 연동:** Nexon Open API의 실제 응답(H-01), Gemini 모델명 유효성(CFG-01), Render 설정(빌드/시작 명령, 워커 수, 프록시 헤더), Supabase 실제 스키마·권한.
- **추가 검증이 필요한 항목:** SEC-01, OPS-01, H-01, DEAD-01/04/05/06/07/08(전체 grep + 테스트 참조), 번들 크기, `/api/character`·`/api/history`의 실측 지연(구간별 계측), `static/style.css`와 `react-compat.css`의 중복 범위.
- **기준 브랜치 주의:** 이 문서는 기본 브랜치 HEAD(`259acf6`) 기준이다. `docs/P5_LEGACY_CLEANUP_AUDIT.md`는 별도 리팩터링 브랜치에서 작성되었다고 명시하고 있으므로, 작업 중인 브랜치와 차이가 있으면 해당 항목(특히 DUP-01, STR-04)을 재확인해야 한다.